

Apuntes: Backpropagation

Modulo 4 — Diplomado Superior en Redes Neuronales y Deep Learning

Diana Blanco

1. Contexto: Por que necesitamos backpropagation?

En el forward pass, los datos fluyen de la entrada hacia la salida produciendo una prediccion. Pero la red no sabe que tan equivocada estuvo cada peso. **Backpropagation** calcula exactamente eso: cuanto contribuyo cada peso al error total, usando la **regla de la cadena** del calculo diferencial.

■ **Nota de clase:** Analogia (videojuegos): es como el replay de una partida perdida. Ves exactamente en que momento fallaste (error en la salida), luego remontas que decision te llevo ahi (gradiente de cada capa), y ajustas tu estrategia (actualizar pesos) para la siguiente partida.

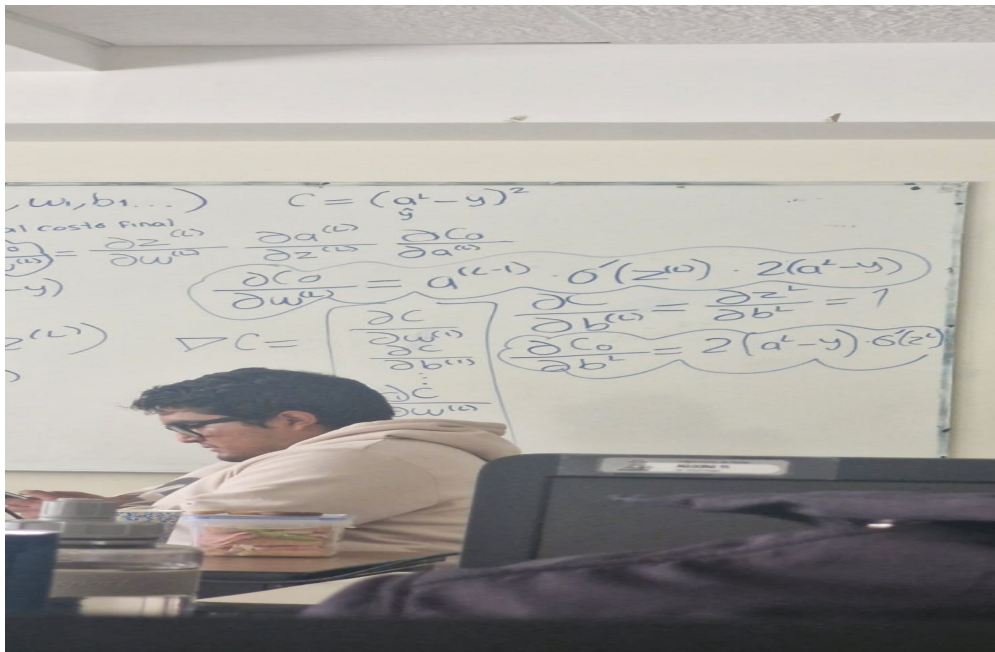


Figura 1: Pizarra de clase — Forward pass y estructura basica de la red

2. Notacion utilizada en clase

Simbolo	Significado
L	Indice de la ultima capa (capa de salida)
l	Indice de una capa arbitraria (1, 2, ..., L)
$w^{(l)}$	Peso de la capa l
$b^{(l)}$	Sesgo de la capa l
$a^{(l)}$	Activacion de la capa l (salida de esa capa)

Simbolo	Significado
$z^{(l)}$	Suma ponderada de la capa l (antes de sigmoid)
y	Valor real (etiqueta del ejemplo)
C	Funcion de costo total
C_0	Costo para un solo ejemplo
$\sigma(z)$	Funcion sigmoid: $1 / (1 + e^{-z})$
$\sigma'(z)$	Derivada de sigmoid: $\sigma(z) * (1 - \sigma(z))$

■ Consejo: aprendete bien la notacion antes de seguir. Cada simbolo aparece una y otra vez en las formulas de backpropagation.

3. Forward Pass — Base del calculo

Antes de calcular derivadas, recordamos como fluyen los datos hacia adelante. Para la ultima capa L:

$$z^{(L)} = w^{(L)} * a^{(L-1)} + b^{(L)}$$

$$a^{(L)} = \text{sigma}(z^{(L)})$$

Donde $a^{(L-1)}$ es la activacion de la penultima capa (la entrada a la ultima capa). Este es el patron que se repite en cada capa.

■ En codigo (Network_1.ipynb): `z = sum(w_i * x_i for w_i, x_i in zip(weights, inputs)) + bias`

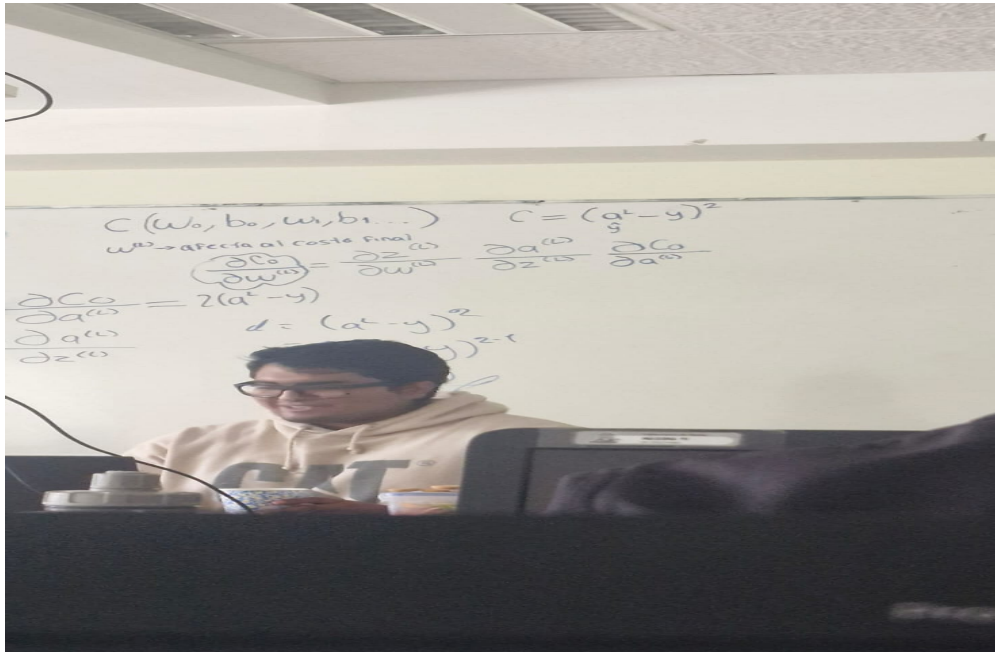


Figura 2: Pizarra de clase — Regla de la cadena y derivadas parciales

4. Funcion de Costo MSE

La funcion de costo mide que tan equivocada esta la prediccion. Usamos el **Error Cuadratico (MSE)** para un solo ejemplo:

$$C_0 = (a^{(L)} - y)^2$$

El costo total C es el promedio sobre todos los ejemplos de entrenamiento. El cuadrado garantiza que el costo sea siempre positivo y penaliza mas los errores grandes.

■ **Nota de clase:** El profe enfatizo: la funcion de costo depende de **TODO**s los parametros de la red. Se denota como $C(w_0, b_0, w_1, b_1, \dots)$ porque cada peso y sesgo afecta el resultado final.

■ En codigo (Network_1.ipynb): `mse_loss(pred, target) = (pred - target)^2`. La derivada $dC/da = 2 * (a - y)$ se usa en el paso 4 de backprop.

5. Regla de la Cadena — Corazon de Backpropagation

Para saber cuanto afecta el peso $w^{(L)}$ al costo final, aplicamos la **regla de la cadena** del calculo diferencial. El costo no depende directamente de $w^{(L)}$, depende de $z^{(L)}$, que depende de $a^{(L)}$, que depende de C:

$$dC/dw^{(L)} = dz^{(L)}/dw^{(L)} * da^{(L)}/dz^{(L)} * dC/da^{(L)}$$

Cada fraccion es una derivada parcial. Las calculamos por separado:

Derivada	Calculo	Resultado
$dC/da^{(L)}$	Derivada de $(a-y)^2$ respecto a a	$2 * (a^{(L)} - y)$
$da^{(L)}/dz^{(L)}$	Derivada de $\sigma(z)$ respecto a z	$\sigma'(z^{(L)}) = \sigma * (1 - \sigma)$
$dz^{(L)}/dw^{(L)}$	Derivada de $(w*a_{prev} + b)$ respecto a w	$a^{(L-1)}$
$dz^{(L)}/db^{(L)}$	Derivada de $(w*a_{prev} + b)$ respecto a b	1

■ Cada operacion en el codigo (suma, multiplicacion, sigmoid) tiene su propia version de la regla de la cadena implementada en `_backward()`.

6. Formulas Finales del Gradiente

Gradiente respecto al PESO $w^{(L)}$:

$$dC/dw^{(L)} = a^{(L-1)} * \sigma_{\text{prime}}(z^{(L)}) * 2*(a^{(L)} - y)$$

Gradiente respecto al SESGO $b^{(L)}$:

$$dC/db^{(L)} = 1 * \sigma_{\text{prime}}(z^{(L)}) * 2*(a^{(L)} - y)$$

$$\text{Simplificado: } dC/db^{(L)} = 2*(a^{(L)} - y) * \sigma_{\text{prime}}(z^{(L)})$$

■ **Nota de clase:** Nota: $dz/db = 1$, entonces el gradiente respecto al sesgo es identico al del peso pero sin el factor $a^{(L-1)}$. Esto explica por que los sesgos aprenden diferente a los pesos.

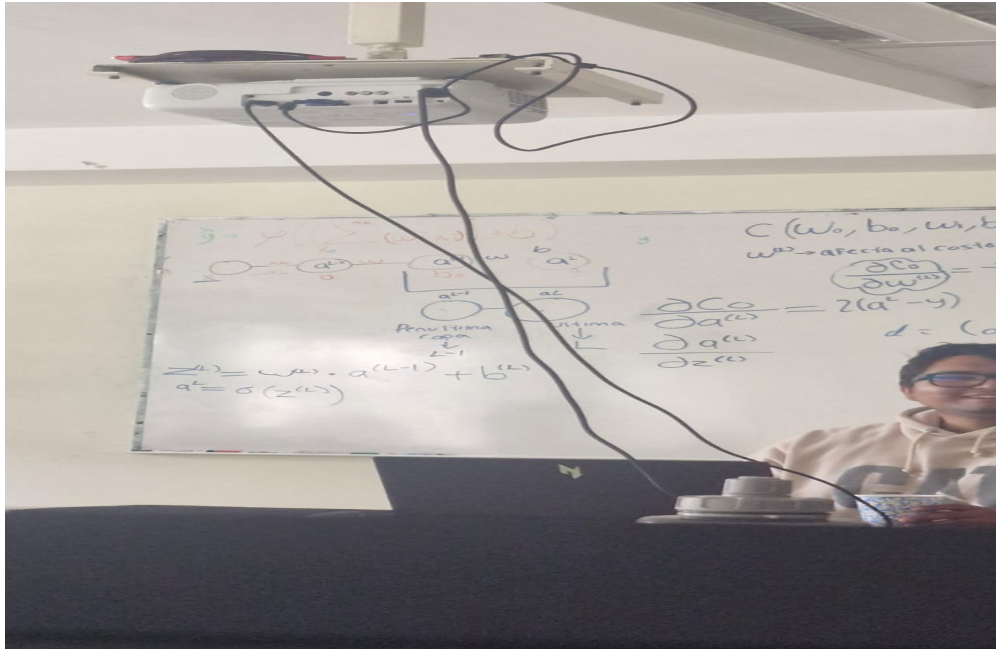


Figura 3: Pizarra de clase — Backpropagation completo en la red 2-2-1

7. El Gradiente como Vector (Nabla C)

El gradiente total de la red es un vector que contiene las derivadas parciales respecto a TODOS los parametros:

$$\text{nabla} C = [dC/dw^{(1)}, dC/db^{(1)}, dC/dw^{(2)}, dC/db^{(2)}, \dots]$$

Este vector apunta en la direccion de **maxima subida** del costo. Al restarlo (multiplicado por el learning rate), nos movemos cuesta abajo:

$$w_{\text{nuevo}} = w_{\text{viejo}} - lr * (dC/dw)$$

■ En codigo (Network_1.ipynb): `p.data -= learning_rate * p.grad` donde $p.grad = dC/dp$

8. Derivada de Sigmoid — Por que es especial

La derivada de sigmoid tiene una propiedad muy conveniente: se puede expresar en terminos de la misma sigmoid, sin recalcular:

$$\sigma(z) = 1 / (1 + e^{-z})$$

$$\sigma'(z) = \sigma(z) * (1 - \sigma(z))$$

Esto significa que si ya calculamos $\sigma(z)$ en el forward pass, reutilizamos ese valor directamente en el backward pass. No hace falta recalcular nada.

z	$\sigma(z)$	$\sigma'(z)$	Interpretacion
-10	~0.000	~0.000	Neurona inactiva — gradiente casi nulo
-2	0.119	0.105	Poco activa
0	0.500	0.250	Maximo de σ' — aprende mas rapido aqui
+2	0.881	0.105	Muy activa
+10	~1.000	~0.000	Neurona saturada — gradiente casi nulo

■ **Nota de clase: Problema de saturacion (Vanishing Gradient):** cuando z es muy positivo o muy negativo, $\sigma'(z) \sim 0$ y el gradiente desaparece. Este es uno de los problemas historicos de las redes profundas con sigmoid. Funciones como ReLU lo mitigan (derivada = 1 para $z > 0$).

■ En codigo (Network_1.ipynb): la derivada se implementa como: `self.grad += (s * (1 - s)) * out.grad` donde $s = \sigma(z)$

9. Algoritmo Completo de Backpropagation

Paso	Operacion	Formula /Codigo
1	Forward pass	Para cada capa l: $z^l(l) = w^l(l) * a^{l-1} + b^l(l)$, $a^l(l) = \text{sigma}(z^l(l))$
2	Calcular costo	$C = (a^L(L) - y)^2$ (MSE para 1 ejemplo)
3	Gradiente salida	$dC/da^L(L) = 2 * (a^L(L) - y)$
4	Backprop ultima capa	$dC/dw^L(L) = a^{L-1} * \text{sigma_prime}(z^L(L)) * 2 * (a^L(L) - y)$
5	Backprop capas ocultas	Aplicar regla de la cadena hacia atras en cada capa
6	Actualizar pesos	$w = w - lr * dC/dw$ (descenso por gradiente)
7	Repetir	Volver al paso 1 con el siguiente batch / epoch

10. Correspondencia: Pizarra → Código (Network_1.ipynb)

La clase **Value** del notebook implementa exactamente estas derivadas. Cada operacion define su propio `_backward()` con la regla de la cadena correspondiente:

Matematica (pizarra)	Codigo (Network_1.ipynb)
$dC/da^L(L) = 2 * (a - y)$	<code>mse_loss: diff = pred - target; return diff * diff</code>
$\text{sigma}'(z) = s * (1 - s)$	<code>Value.sigmoid._backward: self.grad += (s * (1 - s)) * out.grad</code>
$dz/dw = a^{L-1}$	<code>Value.__mul__._backward: self.grad += other.data * out.grad</code>
$dz/db = 1$	<code>Value.__add__._backward: self.grad += out.grad</code>
$w = w - lr * dC/dw$	<code>for p in net.parameters(): p.data -= lr * p.grad</code>
zero_grad antes de cada paso	<code>net.zero_grad(): p.grad = 0.0</code>

11. Notas finales de clase

■ **Nota de clase:** La red con pesos aleatorios da ~17-50% de accuracy (depende del problema). **Despues de entrenar con backpropagation llega al 100%.** La diferencia es el entrenamiento.

■ **Nota de clase:** El orden de las operaciones es crucial: `forward -> zero_grad -> backward -> step`. Si cambias el orden, el entrenamiento falla. En PyTorch es el mismo orden: `pred = model(X); loss = criterion(pred, y); optimizer.zero_grad(); loss.backward(); optimizer.step()`

■ **Nota de clase:** Para redes profundas, el vanishing gradient hace que las primeras capas aprendan muy lentamente. Soluciones: ReLU (derivada = 1 para $z > 0$), Batch Normalization, ResNet (skip connections).